

Electro Pi | QA Automation Technical Assessment

Part 1: UI Test Automation & Architecture

1. Framework Choice

- I chose Selenium because of its true cross browsers/cross platform coverage and Java plus TestNG fits naturally with existing enterprise QA stacks and brings annotation-based test structuring, built-in parallel execution, test grouping (smoke vs. regression), and rich XML-based suite configuration, which matter as the POS regression suite grows.
- Selenium also integrates cleanly with established enterprise tooling: Maven for build/dependency management, TestNG/Extent Reports for reporting, and CI platforms such as GitHub Actions or Jenkins.

2. Design Pattern

- I used Page Object Model (POM) to help me separating concerns into distinct, single-responsibility layers rather than letting page classes accumulate both locators, actions, and wait logic:
 - I created a **Driver Factory** to achieve thread-safe (ThreadLocal-based) WebDriver factory, so tests can run in parallel via TestNG without sharing driver instances across threads.
 - Also created **config.properties** to save system keys like url, username, password, default browser,....etc., then created **Config Reader** as a utility to expose a named, type-safe accessor method for every key in config.properties (e.g. baseUrl(), browser(), headless(), Admin.username()), so no raw string keys are scattered across the codebase, and a typo fails at compile time rather than silently at runtime.
 - Utilities folder also contains **Wait Utils**, **CSV Reader** and **ScreenshotUtils**: **WaitUtils** is the single source of truth for every explicit wait in the framework (visibility, clickability, invisibility of spinners, URL changes). No other class builds its own WebDriverWait.
 - Then I created **BasePage** to hold only reusable page ACTIONS: click, type, getText, isVisible. It implements no wait mechanics of its own; every action delegates to WaitUtils internally.
 - After setting up the structure, I created **Page classes (LoginPage, InventoryPage, AdminHomePage)** extend BasePage and hold only their own

By locators plus page-specific business methods (e.g. loginAs(), addItem()). They never reference WaitUtils directly for standard actions — only for page-specific waits BasePage doesn't cover, such as waiting for a loading spinner to disappear.

- Then I created **BaseTest** class to carry test hooks which happens after and before tests
 - Then I put **DataProviders** and **CSV test data** as data-driven tests that read their input rows from external CSV files via a TestNG **@DataProvider**, so new test cases can be added by editing a spreadsheet, not the Java code.
 - Then added **Listeners (TestListener + ExtentReportManager)** which is hooked into the TestNG lifecycle to auto-generate an HTML report and attach a screenshot on every failure, without any manual reporting code inside the tests themselves.
 - Then I added **tests** in a separate test class.
- This layering gives scalability (new pages/tests are added independently of the wait/config/reporting layers), reusability (every page shares the same actions and wait logic), maintainability (a broken selector, a flaky wait, or a wrong config value each has exactly one place to fix), and easy debugging (failures are traceable to a single, narrow-responsibility class rather than a monolithic page object).

3. Implementation

It is attached via GitHub through this repo:

[MohsenHagag/ElectroPiAssessment: Required Assessment for applying QA Automation Engineer Position for ElectroPi](#)

4. Stability

Flaky tests are avoided without any hardcoded sleeps by:

- Centralizing all waits in WaitUtils, using WebDriverWait + ExpectedConditions exclusively — no Thread.sleep() anywhere in the framework.
- Waiting for actual conditions tied to the async event, not fixed durations: visibilityOfElementLocated before reading/interacting with an element, elementToBeClickable before every click, invisibilityOfElementLocated to let loading spinners clear before proceeding, and urlContains to confirm navigation completed.
- Using resilient locators (stable IDs/classes) rather than brittle CSS/XPath tied to DOM structure that changes often in dynamic, componentized UIs.
- Isolating test data via CSV-driven data providers, so parallel test runs don't depend on shared or leftover state.

- Capturing a screenshot automatically on any failure via the TestListener, so a flaky-looking failure can be diagnosed from the report without needing to reproduce it manually.
- Running via TestNG's thread-safe parallel execution (ThreadLocal WebDriver in DriverFactory), so concurrent tests never interfere with each other's browser state.
- Reserving automatic retries strictly for genuine environment-level flakiness (e.g. CI runner hiccups) rather than using retries to mask real, reproducible failures.

Part 2: API Testing Strategy

1. Test Scenarios

ID	Input	Expected Status	Expected Result
P1	Valid payload with all required fields (unique sku, valid category_id, positive quantity/price)	201 Created	Response returns the created item with a generated id; the sku in the response matches the request; item is retrievable via a subsequent GET.
P2	Valid payload with quantity: 0 (boundary — adding a new item with zero starting stock)	201 Created	Item is created successfully with quantity: 0, confirming the boundary value is accepted rather than rejected.
N1	Duplicate sku (“MS-001” already exists for this tenant)	409 Conflict	Request rejected with a clear error message (e.g. “sku already exists”); no duplicate row created.
N2	Missing required field (e.g. no item_name)	400 Bad Request	Request rejected with a validation error naming the missing field; no item created.
N3	Missing, malformed, or expired Authorization token	401 Unauthorized	Request rejected before reaching business logic; no item created; error response does not leak internal details.

2. Postman Automation

- I created a new collection and environment to store it in a Postman variable rather than hardcoding it then added needed variables **baseUrl** and **SKU with** assumed values and **authToken** without value.
- At the collection, I created two requests **login** and **create product**: at login request body, i added assumed credentials and at headers I selected content type as application/json then, at tests tab i added a code to convert the JSON response into a JavaScript object then stores token inside authToken.
- At create product request, I selected Authorization type as Bearer Token and entered at token field {{authToken}} to relate with it as a variable and at request body, i selected it as raw then json and added specified json data as requested from the task and at request tests, I added code to ensure that request must return exactly HTTP status 201, And if the API returns only 201, test passes.
- Then added code to check that SKU is returned then compare it with the expected, if they are identical, test passes.

The assertion code you would write in the Tests tab to validate a successful 201 Created response and verify the sku returned in the response body. (Is provided at [github repo](#))

```
pm.test("Status code is 201 Created", function () {  
    pm.response.to.have.status(201);  
});
```

```
const response = pm.response.json();
```

```
pm.test("SKU is returned in the response", function () {  
    pm.expect(response).to.have.property("sku");  
});
```

```
const expectedSku = pm.variables.get("sku");
```

```
pm.test("Returned SKU matches requested SKU", function () {  
    pm.expect(response.sku).to.eql(expectedSku);  
});
```

Part 3: CI/CD & Database validation

1. Database Testing

SQL Query (Is provided at [github repo](#)):

```
SELECT
  p.id,
  p.item_name,
  p.sku,
  p.quantity,
  p.price,
  c.category_name
FROM products p
JOIN categories c
  ON p.category_id = c.id
WHERE p.sku = 'SKU-12345'
  AND c.category_name = 'Electronics';
```

2. CI/CD Integration

- The UI and API suites are integrated into a single GitHub Actions workflow (.github/workflows/ci.yml), triggered on every push and pull request to main, structured as two jobs:
 - Api-tests: installs Node.js, installs Newman (Postman's CLI runner) plus the HTML reporter, and runs the exported Postman collection headlessly. The HTML report is uploaded as a build artifact regardless of pass/fail, so results are downloadable from the Actions run summary.
 - Ui-tests: declared with needs: api-tests, so it only runs once the API layer is confirmed healthy. It sets up JDK 11, caches Maven dependencies (keyed on pom.xml's hash) to speed up repeat runs, and executes the Selenium/TestNG suite with mvn test -Dheadless=true. Because ConfigReader reads System.getProperty before falling back to config.properties, this single flag switches the whole suite to headless Chrome in CI with no code changes, while still running headed locally by default. GitHub's ubuntu-latest runners already have Chrome installed, and WebDriverManager (used in DriverFactory) auto-downloads the matching ChromeDriver version at runtime, so no manual browser/driver setup is needed in the pipeline.
- Both jobs upload their reports (Newman's HTML report and the TestNG/Extent report) as build artifacts with if: always(), so results are available for inspection even when a job fails. Credentials that are genuinely sensitive (as opposed to the assumed test data used in this assessment) would be stored as GitHub Actions repository secrets and injected via environment variables, never committed in plaintext.

3. Execution Strategy

To keep feedback fast while still getting full coverage, the pipeline is intentionally staged rather than running everything, every time, at the same depth:

- API tests run first, on every push/PR. They have no browser overhead and typically finish in seconds, so they give the fastest possible signal on backend regressions and gate the rest of the pipeline via needs: api-tests.
- A smoke subset of the UI suite (TestNG group "smoke") is designed to run on every push/PR alongside the API layer — covering only the critical happy paths (e.g. login, add product, success toast) — to catch major UI breakages early without slowing down every commit with the full browser suite.
- The full UI regression suite (TestNG group "regression", including the CSV-driven data provider covering multiple valid/invalid inputs) is intended to run on merge to main and on a nightly schedule, since it is slower and less time-critical than per-commit feedback.
- This tiering means a broken backend or a broken critical UI path is caught within minutes on every commit, while the more expensive, comprehensive checks still run regularly without adding friction to normal development velocity.